

Firmware Upgrade System Design for ATmega328PB

Objective

Design a safe and user friendly firmware upgrade mechanism for an ATmega328PB based device that allows a non technical user to update firmware through a wired connection using a computer and a downloadable upgrade tool.

The user workflow should be:

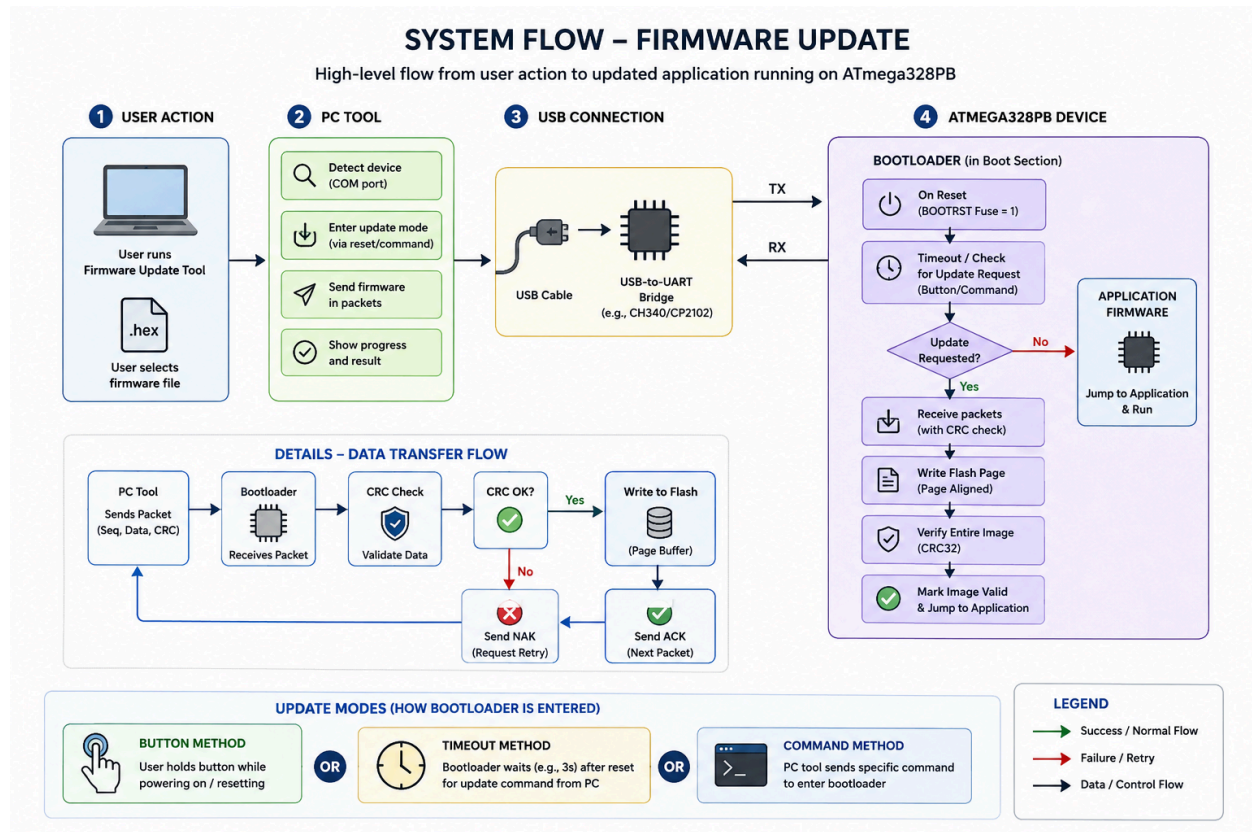
- Download and install upgrade tool
- Connect device with USB cable
- Open tool and select firmware
- Click update
- Wait for completion

System Overview

The system consists of:

- ATmega328PB microcontroller
- Bootloader stored in protected flash memory
- USB to UART hardware interface
- PC based firmware upgrade application
- Serial communication protocol

System flow



Application Firmware

Hardware Architecture

ATmega328PB does not support native USB. Therefore the design includes a USB to Serial converter(eg CH340)

The converter connects:

USB <-> USB UART Bridge <-> MCU RX/TX pins

This allows communication between the PC tool and microcontroller through a virtual COM port.

Bootloader Design

A bootloader is permanently stored in the protected boot section of flash memory.

Responsibilities:

- Run immediately after reset
- Determine whether firmware update mode should start
- Receive firmware packets
- Validate received data
- Program flash memory page by page
- Verify final firmware image
- Start application firmware

Fuse Settings and Boot Configuration

Fuse settings are programmed during factory production through the standard ISP interface and remain fixed during normal operation.

BOOTRST = 0 (Programmed)

Configures the reset vector to start execution from the bootloader section after any reset event instead of the application reset vector. This guarantees that firmware update logic executes before the application firmware.

BOOTSZ[1:0]

Configured to reserve sufficient flash memory for the bootloader implementation (approximately 1 KB target allocation). The selected value must match the ATmega328PB boot section size table.

Boot Lock Bits

Boot lock protection is enabled to prevent the application firmware from modifying or overwriting bootloader memory.

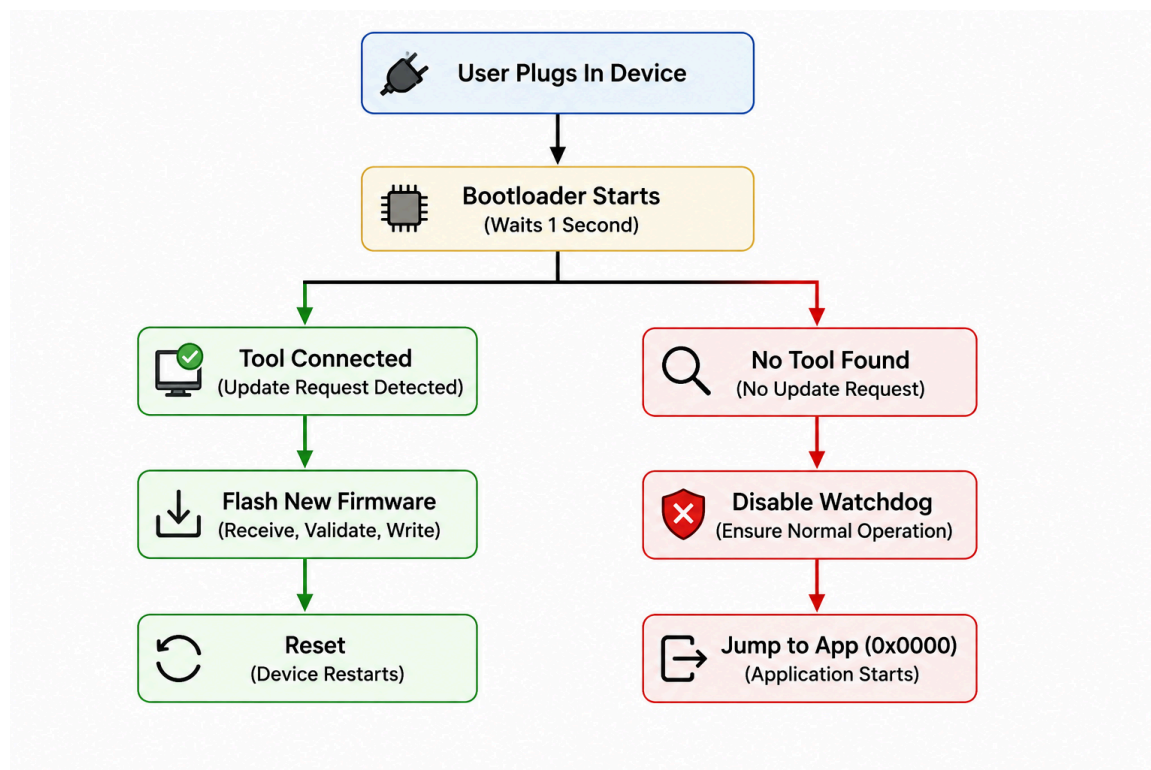
Example:

```
BLB11 = 1
BLB12 = 0
```

This ensures:

- Bootloader remains protected during normal runtime
- Application firmware cannot corrupt update functionality
- Device recovery remains possible even after application failures

Timeout Based Bootloader Entry



Firmware Upgrade Process

Step 1: User connects device through USB cable

Step 2: PC application automatically detects device

Step 3: Device enters bootloader mode

Step 4: Firmware file is sent in packets

Step 5: Bootloader:

- receives packet • verifies CRC • writes flash page • sends acknowledgment

Step 6: Entire image is verified

Step 7: Application starts

Execution Fallback: Jump immediately to the existing user application if no valid computer handshake occurs within the timeout window

Flash Page Alignment

ATmega328PB flash memory is page-based and cannot be programmed arbitrarily byte-by-byte.

Bootloader process:



Example: Page buffering prevents partial writes and improves reliability.

Bytes 0-127 → Page 0

Bytes 128-255 → Page 1

Bytes 256-383 → Page 2

Fault Handling

Corrupted packet

- Reject packet
- Request retransmission

Cable unplugged during update

- Abort update safely
- Remain in bootloader on next reset

Invalid application image

- Do not execute application firmware
- Stay in bootloader mode

CRC mismatch

- Abort firmware update process
- Notify host and restart update sequence if required

Automatic Bootloader Entry (External Trigger)

The system supports automatic entry into bootloader mode without requiring a physical button press.

The bootloader is triggered through a controlled external mechanism, such as:

- A specific UART command sent from the PC upgrade tool
- A predefined reset sequence initiated via USB-to-UART control lines (e.g., DTR toggle)
- A watchdog or timeout condition after reset if no valid application start signal is detected

Behavior:

- On reset, the bootloader starts execution first
- If a valid “enter update mode” trigger is detected, the device remains in bootloader mode
- Otherwise, it jumps to the application firmware

This approach ensures a fully user-transparent firmware update process while eliminating the need for manual intervention.

Summary

The design uses a protected UART bootloader with a USB-to-serial interface and a simple PC application that enables reliable field firmware updates for non-technical users while providing safeguards against interrupted or corrupted upgrades.